

We are solving Timestamped Secrets, a medium cryptography challenge where we must decrypt a message that was encrypted using AES in ECB mode.

I first download the provided message file I need to decrypt and the source code of the script that encrypted the message. I open the message file using the 'cat' command and this is what I got:

```
(hackingmastermind@ctfdestroyingMACHINE)~[~/Downloads]
$ cat message.txt
Hint: The encryption was done around 1770242637 UTC
Ciphertext (hex): 030ea2b59ea3cad39da9dfff761acc2598161c602243e2b0e9e571cee82
85b87
```

There is a hint provided and it has to do with the time the message was encrypted. I keep that in mind. I then opened source code to see how the encryption was done:

```
(hackingmastermind@ctfdestroyingMACHINE)~[~/Downloads]
$ cat encryption.py
from hashlib import sha256
import time
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

def encrypt(plaintext: str, timestamp: int) -> str:
    timestamp = int(time.time())
    key = sha256(str(timestamp).encode()).digest()[:16]
    cipher = AES.new(key, AES.MODE_ECB)
    padded = pad(plaintext.encode(), AES.block_size)
    ciphertext = cipher.encrypt(padded)
    return ciphertext.hex()

if __name__ == "__main__":
    plaintext = "picoCTF{ ... }"
    result = encrypt(plaintext, key)
    print(f"Hint: The encryption was done around {timestamp} UTC\n")
    print(f"Ciphertext (hex): {ciphertext.hex()}\n")

(hackingmastermind@ctfdestroyingMACHINE)~[~/Downloads]
$
```

After reading the source code, I know I have the 'timestamp' value (which is printed), which means I can derive the key. I created a simple script to do that and used an online Python IDE (to avoid having to download the libraries I don't have):

</> Python Code

Reset

Run

```
1 # Python Cryptography Compiler
2 from hashlib import sha256
3 import time
4 from Crypto.Cipher import AES
5 from Crypto.Util.Padding import pad
6
7 timestamp = 1770242637
8
9 key = sha256(str(timestamp).encode()).digest()[:16]
10
11 cipher = AES.new(key, AES.MODE_ECB)
12
13 print(key)
14
```

>_ Output Ready

Copy

Download

Print

```
b'\x90N\xa9\x1dKdCf\x17a\xd2\x13\n\x16"\x80'
```

Now that I have the key, I can easily decrypt the message. This is because AES is a symmetric key algorithm, meaning it uses the same key to encrypt and decrypt. The `Crypto.Cipher` library has a `decrypt()` function that I can use to do the opposite of the `encrypt()` function used in the source code. After running this script, I get the flag:

</> Python Code

Reset

Run

```
1 # Python Cryptography Compiler
2 from hashlib import sha256
3 import time
4 from Crypto.Cipher import AES
5 from Crypto.Util.Padding import pad
6
7 timestamp = 1770242637
8
9 key = sha256(str(timestamp).encode()).digest()[:16]
10
11 cipher = AES.new(key, AES.MODE_ECB)
12
13 ciphertext = bytes.fromhex(
14     ('030ea2b59ea3cad39da9dfff761acc2598161c602243e2b0e9
15     e571cee8285b87')
16 )
17 plaintext = cipher.decrypt(ciphertext)
18
19 print(plaintext)

```

>_ Output Ready

Copy

Download

Print

```
b'picoCTF{sa3S_sEc9t_fbcf37a3}\x04\x04\x04\x04'
```

flag: picoCTF{sa3S_sEc9t_fbcf37a3}

